Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 183 (2021) 526–532

Procedia Computer Science

www.elsevier.com/locate/procedia

10th International Conference of Information and Communication Technology (ICICT-2020)

Improvement of data compression technology for power dispatching based on run length encoding

Jiawei Zhang[a,b,*], Dawei Sun[a]

[a]*Shenyang Institute of Computing Technology,Chinese Academy of Sciences,Shenyang 110168 ,China*
[b]*University of Chinese Academy of Sciences,Beijing 100049,China*

Abstract

Data compression is of great significance to the storage of power dispatch data. The compressed data not only saves a lot of space, but also improves the speed of data retrieval. Therefore, this paper proposes an improved algorithm based on run length encoding to compress power dispatch data. In the case of high data similarity, run length coding and dictionary coding have good compression effects, so this article classifies power dispatch data, and uses an improved run length coding algorithm to compress data with high similarity. For similarity Low data is compressed using the Lz4 algorithm that comes with HBase, and a Base-128 Varints encoding the number of occurrences of elements in the run-length encoding is proposed to further improve the compression rate of similar data. This improved algorithm is proven in experiments In some scenarios, the compression rate is indeed improved.

Keywords: Run Length Encoding; Compression; Base-128 Varints;

1. Introduction

As the country continues to develop rapidly, the power system of the State Grid can be said to be increasingly large, accompanied by problems such as the transmission and storage of massive power dispatch data, so it is also a necessary choice to compress data in a limited space. In fact, the research on data compression was proposed by W.F Sheppards in the experiment "Rounding Real Numbers to Decimal Numbers" as early as the end of the

* Corresponding author. Tel.: +86-18510626786
E-mail address: zjw_alex@163.com

10.1016/j.procs.2021.02.093

eighteenth century. Then a hundred years later, the first practice of data compression-Morse Code. Until today, the exploration of data compression has not stopped, but the compression technology proposed in the era when there was no computer still has great application value. For example, Huffman coding has always occupied an important position, so there are continuous improvements based on Huffman coding. Algorithm is proposed. With the widespread application of data storage in power systems, a large amount of raw data is generated. The high storage cost and inefficient search speed make data compression algorithms more and more needed. Therefore, this paper proposes an improved compression algorithm. Base-128 Varints is an algorithm for encoding numeric types, which can reduce the volume of serialized data. This article uses the number of occurrences of elements in the run-length encoding to improve the compression rate of the run-length encoding. For example, "a5b3" can be used for "aaaaabbb". For run length coding, we can know that it has a better compression effect under the condition of high repetition. Therefore, first classify power dispatch data. This article mainly focuses on improving the compression method for data with higher similarity.

2. HBase Compression Algorithm

2.1. File storage

The Fig. 1 shows the structure of the table in HBase.

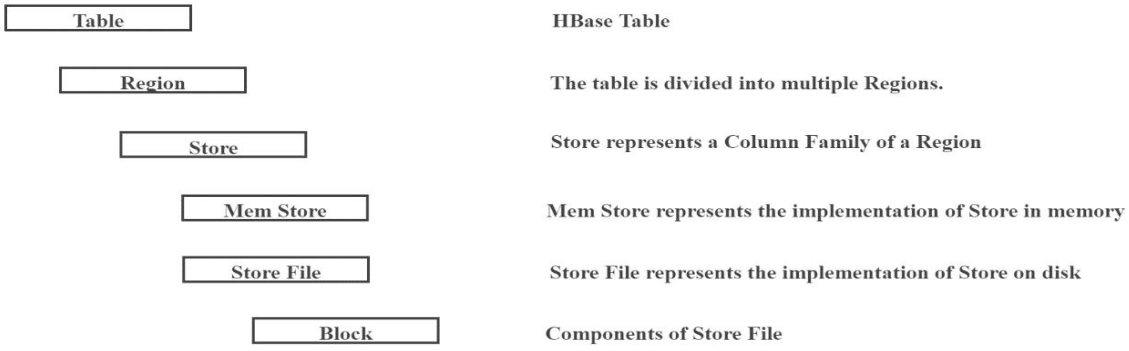

Fig. 1. HBase Structure.

2.2. Introduction To HBase Compression Algorithms

At present, the compression algorithms in HBase include GZip, Lzo, Snappy, Zstd, and Lz4. Among them, Luo Yanxin introduced the Gzip and Lzo algorithms in [1-3], so the following three algorithms will be introduced below.

Snappy algorithm. Snappy is a high-speed compression algorithm, which is characterized by fast compression speed. The table 1 shows that the comparison between this algorithm and the previous two algorithms in compression ratio, encoding ratio and decoding ratio.

Table 1. Snappy algorithm compared with other algorithms

Algorithm	%remaining	Encoding	Decoding
Snappy	22.2%	172MB/s	409MB/s
Gzip	13.4%	21MB/s	118MB/s
Lzo	20.5%	135MB/s	410MB/s

Zstd algorithm. Zstd is a lossless compression algorithm. Because of its fast compression speed and high compression ratio, it is often used for real-time data compression, which cannot be replaced by other algorithms.

Lz4 algorithm. Lz4 is also a lossless compression algorithm, an algorithm based on Lz77, which pursues the ultimate compression ratio.

Table 2. Lz4 algorithm compared with other algorithms

Compressor	Ratio	Compression	Decompression
Lz4 fast 8	1.799	909MB/s	3260MB/s
Lz4 default	2.101	612MB/s	3120MB/s
Lzo 2.09	2.108	609MB/s	835MB/s

It can be seen that the Lz4 algorithm is better than the Lzo algorithm in all aspects.

3. Improve Length Encoding

3.1. Base-128 Varints Encoding

Protocol Buffer is a data description language developed by Google, similar to XML, JSON and other formats, but the serialized data volume is more than tens of times smaller than XML and JSON, so it is widely used. Protocol Buffer uses the Base-128 Varints encoding method, which reduces the serialized data volume and improves the transmission speed. Base-128 Varints is an algorithm for encoding numeric types. For example, for integer types, from the minimum to 0 to the maximum, only the number of bytes occupied by MAX_INT is the same, but for 0, 1, etc., it is wasted Space, and Base-128 Varints uses a dynamic representation method to represent values. Smaller numbers occupies less bytes, and larger numbers use more space to represent, thus saving space[4-8]. In this encoding method, the highest bit is used as a flag bit to indicate whether the following byte and the byte represent the same value, where 1 means “yes” and 0 means “no”.

For example, when we represent 6, since the binary representation of 6 is 00000110, it can be represented by 7 binary bits, so the highest bit is 0, and one byte can represent 6; another example is the value 600, which is expressed as: 11011000 000000100

The high bit of the previous byte is 1 to indicate that the next byte also belongs to the value, and the high bit of the next byte is 0 to indicate that it is the last byte of the value.

The process of encoding and decoding above is

1) Remove the mark of each byte to get: 1011000 00000100.

2) Reverse the remaining bytes to get: 00000100 1011000.

3) The calculated value is: 512 + 64 + 16 + 8 = 600.

It can be seen that expressing an integer value in the form of a dynamic length can greatly save space.

3.2. Improvement of Length Encoding of Variable Length Integer

Run length coding often adopts “element and appearance position and continuous length” or “element and times of occurrences”. This article adopts the latter.

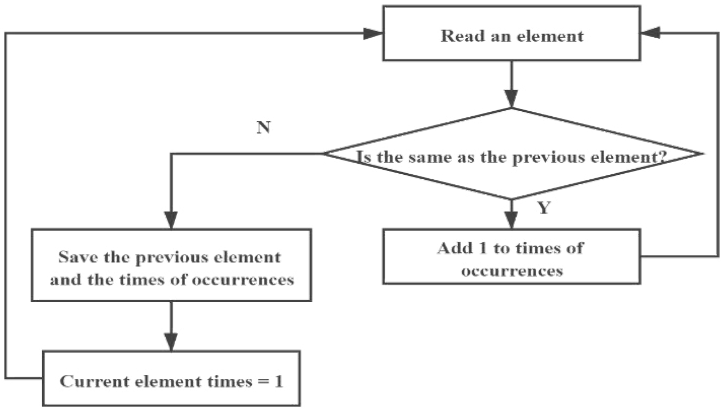

Fig. 2. Run Length Encoding algorithm flow.

For the following data:

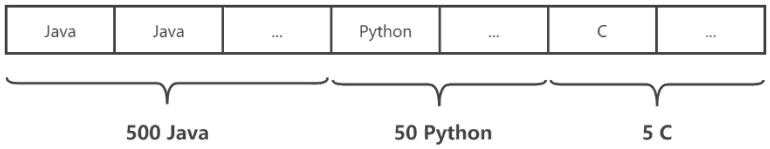

Fig. 3. Source data.

The encoding result is “Java 500 Python 50 C 5”.

Assuming that a character has a size of one byte and an integer has a size of 2 bytes.

The size before encoding is “500*4+50*6+5*1 = 2305”.

The compression rate of this algorithm is “17/2305 = 0.00737527”.

It can be seen that the compression effect is very good, but the algorithm is not suitable for situations where the data similarity is small. Assuming the extreme case is that the source data is data without repeated characters, then the compressed size will be larger than the source data[9-10].

In this paper, the following improvements are made to run length encoding: use element + number of occurrences for encoding, and use Varints encoding to re-encode the number of times when the number of times is greater than 4, so that it can be further compressed. The improved process is as follows:

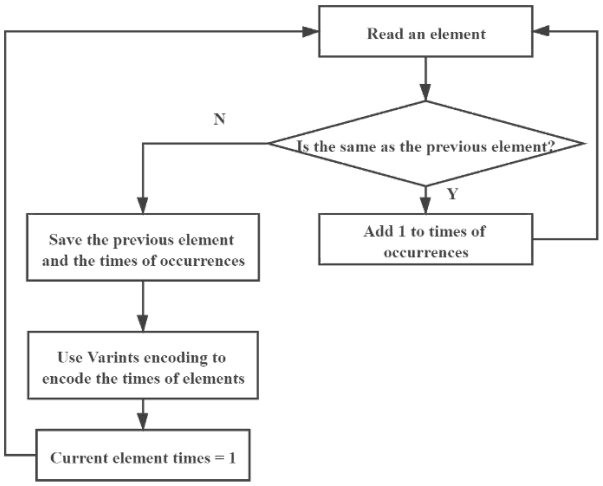

Fig. 4. Flow of the Run Length Encoding with Varints.

For the case of high data repetition, this method has a good compression effect. At the same time, when an integer is used to indicate the number of occurrences, and the number of occurrences of a single character does not exceed 7/8 of the maximum value of the integer, Varints is used for the number of times Further coding can make the compression effect better.

4. Analysis of Experimental Results of Improved Compression Algorithm

The experiment adopts a stand-alone test method to test the compression effect of the improved compression method under the condition of high data similarity. Therefore, the data used in the experiment are source data with high similarity. This article adopts some samples of the power dispatch of the State Grid Northeast Company. The test data is in the table 3.

Table 3. Test data.

Property	Type
Voltage	Double
Max Voltage	Double
Power	Double
Electric Current	Double

Table 4. Lz4 algorithm compared with other algorithms.

Compressor	Ratio	Compression	Decompression
Varints-Run Length Encoding	1.681	642MB/s	2350MB/s
Lz4 fast 8	1.799	909MB/s	3260MB/s
Lz4 default	2.101	612MB/s	3120MB/s
Lzo 2.09	2.108	609MB/s	835MB/s

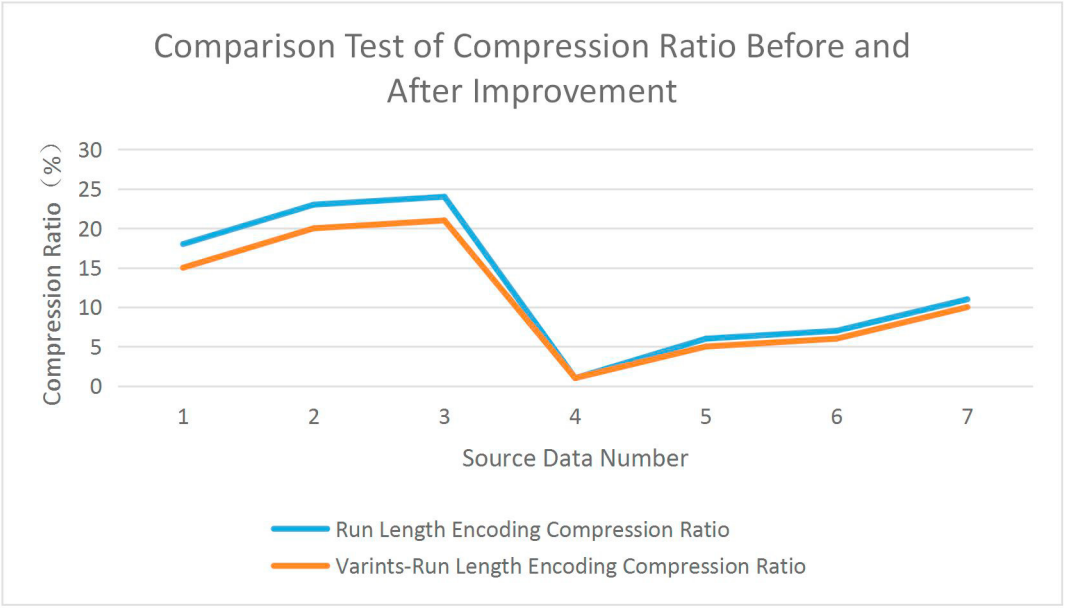

Fig. 5. The test results of the encoding compression ratio.

As can be seen from the above figure, the compression ratio of the improved run length encoding has increased by about 3%. From this, it can be seen that the improved algorithm does effectively increase the compression ratio when the data has high degree of similarity.

Through comparison, it is found that for power dispatch data, the data can be divided into two categories by classification methods such as Bayesian classifiers, one is data with higher similarity, and the other is data with lower similarity. The data with higher similarity can be compressed using improved formation codes, thereby obtaining a higher compression rate. The other data is compressed using the currently adopted scheme, that is, using HBase's Lz4 compression algorithm.

5. Conclusion

Choosing different compression algorithms for compression according to the characteristics of the column storage database is a content worthy of study. The method proposed in this article mainly introduces HBase's own compression algorithm and its compression rate, and then studies Base-128 Varints encoding and run length encoding. Through the analysis of the principle, a method of using Base-128 Varints to perform secondary encoding on the run length code is proposed, and through experimental comparison, it is found that the secondary improvement method does increase the compression rate[11]. In addition, a comparative experiment using the method proposed in this article and using the HBase's own compression algorithm was carried out. From the experimental results, it can be seen that through the classification of data, the method of using different compression algorithms has a better compression rate than the previous methods.

Acknowledgement

This article was completed under the careful guidance of Teacher Sun. Thank you very much, Teacher Sun. Thanks to everyone who helped me.

References

1. Xinyan Luo.Research and implementation of column storage compression algorithm based on HBase[D]. South China University of

Technology, 2011.
2. Liu Chen, Li Yufeng, Chen Hao. Improvement and realization based on LZW lossless data compression technology. Electronic Design Engineering, 2019, 27(24): 51-56.
3. Ke Yan, Meiyi Xie,Hong Zhu. Fixed-length String Compression for Direct Operations in Column-oriented Databases[C]. 2013 Ninth International Conference on Natural Computation (ICNC).China,Shenyang, 2013:1171 - 1176.
4. Tommy Szalapski,Sanjay Madria.On compressing data in wireless sensor networks for energy efficiency and real time delivery[J].Distributed and Parallel Databases,2013,31(2):151 - 182.
5. Mehla, U.S. Inst. of Technol., Nirma Univ., Ahmedabad, India Dasgupta, K.S. Hamming Distance Based Reordering and Columnwise Bit Stuffing with Difference Vector: A Better Scheme for Test Data Compression with Run Length Based Codes[C].VLSID 23rd International Conference on VLSI Design.India,Bangalore, 2010: 33 - 38.
6. Levandoski, J.J. Larson, P.-A. ; Stoica, R. Identifying hot and cold data in main-memory databases[C]. 2013 IEEE 29th International Conference on Data Engineering (ICDE). Australia,Brisbane,2013:26 - 37.
7. Gao Sch. of Inf. Sci. & Eng., Hebei Univ. of Sci. & Technol., Shijiazhuang, China Dongfeng Wang.LSD2H: A Novel Storage Method of Linked Sensor Data Based on HBase[C]. 2014 10th International Conference on Semantics, Knowledge and Grids (SKG).China,Beijing,2014:116 - 119.
8. Shixian Chen, Kai Xu. The application and realization of improved stroke compression coding in information hiding[J]. Coal Technology, 2011, 30(03): 169-170.
9. Bingyun Lv.Analysis and research of commonly used lossless data compression algorithms[J].Electronic World,2019,0(4):5-6.
10. FANJC.Researchondatacompressionandstoragetechnologyofpowersystemcontinuouswaverecorder[D].Qingdao:ChinaUniversityofPetroleum (EastChina),2008:38-39.
11. WANGLJ.Researchandimplementationofdatacompressionandtransmissioninpowersystem[D].Beijing:BeijingUniversityofChemicalTechnology,2010:2-4.